MOTIVATION

Lecture 10 · Transformers and ACT · November 18, 2026

Transformers and ACT

From observations to a sequence of actions

How can a robot combine observations and predict an action chunk: several future motor targets at once?



Mathematics for Robotics · ROB-GY 6013 · NYU Tandon · Jing Zhang · Fall 2026

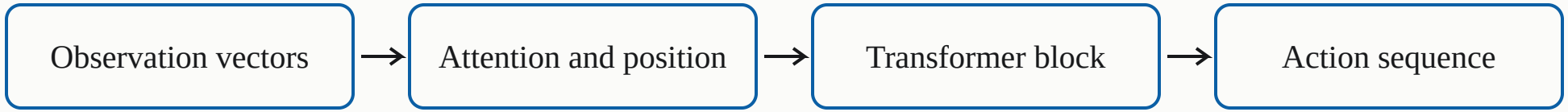
Zhao et al. (2023), §IV-B–C, Appendix C

ROADMAP

Transformers and ACT

The mathematical route

Attention selects information. Position identifies order. A policy turns the resulting representation into actions.



Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Tokens and projections

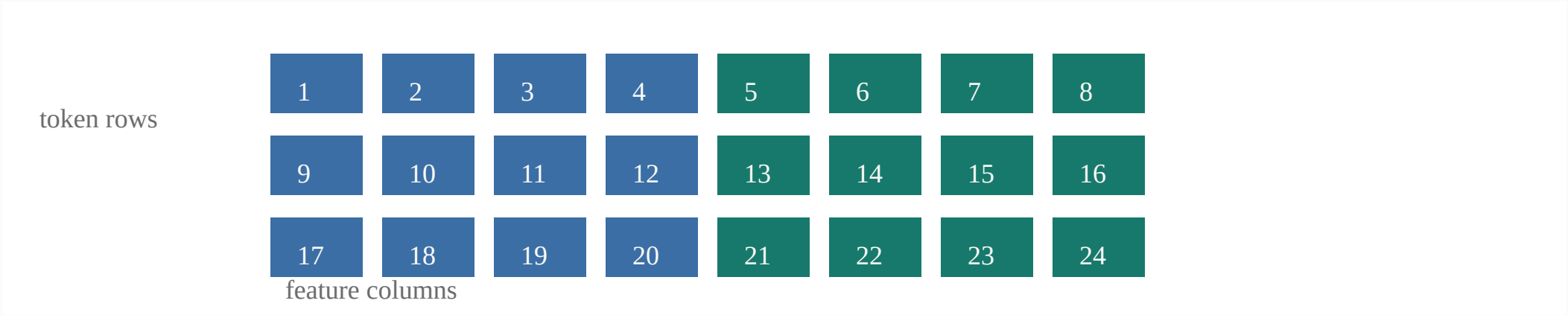
A token is a feature vector

A token represents one item, such as an image patch, with a feature vector.

$$X = [x_1 \ \cdots \ x_T]^T \in \mathbb{R}^{T \times D}$$

x_i column feature vector in $\mathbb{R}^D$

T, D tokens, features per token



Rows identify tokens; columns identify features.

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Tokens and projections

Column weights and row-stored tokens

A query is a learned vector used to compare a token with candidate information.

$$q_i = W_Q x_i \quad \Rightarrow \quad Q = X W_Q^T$$

x_i, q_i	input column in $\mathbb{R}^D$; query column in $\mathbb{R}^{dk}$
W_Q	learned $d_k \times D$ map
X, Q	$T \times D$ inputs; $T \times d_k$ queries

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Tokens and projections

Three learned views of the same input

$$Q = XW_Q^T, \quad K = XW_K^T, \quad V = XW_V^T$$

X	T × D input-token rows
$W_Q, W_K; W_V$	d_k × D maps; d_v × D map
$Q, K; V$	T × d_k queries/keys; T × d_v values

Queries compare with keys. The resulting weights combine values.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXAMPLE

Tokens and projections

A projection computed by hand

$$X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 1 & 3 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & -1 \end{bmatrix}$$

X two tokens with three features

W_Q 2×3 map: three inputs, two outputs

$$Q = XW_Q^T = \begin{bmatrix} 0 & 5 \\ 3 & -1 \end{bmatrix}$$

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Tokens and projections

A new token, the same learned map

$x = (2, -1, 1)^T$, $W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & -1 \end{bmatrix}$. Find $q = W_Q x$.

A $(3, -3)^T$

B $(1, -1)^T$

C A three-coordinate vector

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DEFINITION

Scaled dot-product attention

A score compares a query with a key

A compatibility score compares a query with a key using a scaled dot product.

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}$$

i, j	query index, key index
q_i, k_j	d_k-dimensional columns
s_{ij}	scalar compatibility score



q fixed; key rotates

angle = 2.36 rad

q · k = -0.71

The key direction changes; the query stays fixed.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Scaled dot-product attention

Alignment and length both affect the score

$q = (1, 0)^T$, $k_1 = (1, 0)^T$, $k_2 = (2, 0)^T$, $d_k = 2$. Which key has the larger score?

A Key 1

B Key 2

C They must tie

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DERIVATION

Scaled dot-product attention

All query-key pairs in one matrix

$$S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{T_q \times T_k}$$

- Q Tq × d_k query rows
- K Tk × d_k key rows
- S_{ij} query i versus key j

Self-attention uses one token collection for queries, keys and values. Cross-attention uses one collection for queries and another for keys and values.

queries

keys

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

one scalar per query-key pair

Q = K = $(1,2,3,4)^T$; $d_k = 1$

Each row compares one query with all keys.

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Scaled dot-product attention

Weights combine values

Softmax turns finite real scores into positive weights summing to one within each query row.

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_{\ell=1}^{T_k} e^{S_{i\ell}}}, \qquad y_i = \sum_{j=1}^{T_k} P_{ij} v_j$$

S_{ij}, P_{ij}	scaled score; weight of key j for query i
T_k	number of key-value pairs
v_j, y_i	d_v-dimensional value and output columns

Vaswani et al. (2017), §3.2.1, Eq. 1

EXAMPLE

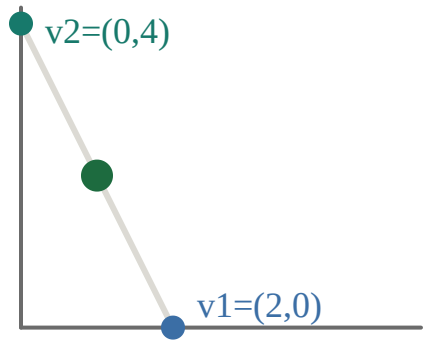
Scaled dot-product attention

Equal scores give an ordinary average

$$s = (0, 0), \quad v_1 = (2, 0)^T, \quad v_2 = (0, 4)^T$$

$s; v_1, v_2$ one query's scores; two value vectors

$p = (1/2, 1/2)$, so $y = (1, 2)^T$.



weight of v2 = 0.50

y = (1.00, 2.00)

Equal weights place the output halfway between the value vectors.

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Scaled dot-product attention

Unequal scores change the mixture

$s = (0, \log 3)$, $v_1 = (2, 0)^T$, $v_2 = (0, 4)^T$. Find the output.

A $(1, 2)^T$

B $(1/2, 3)^T$

C $(0, 4)^T$

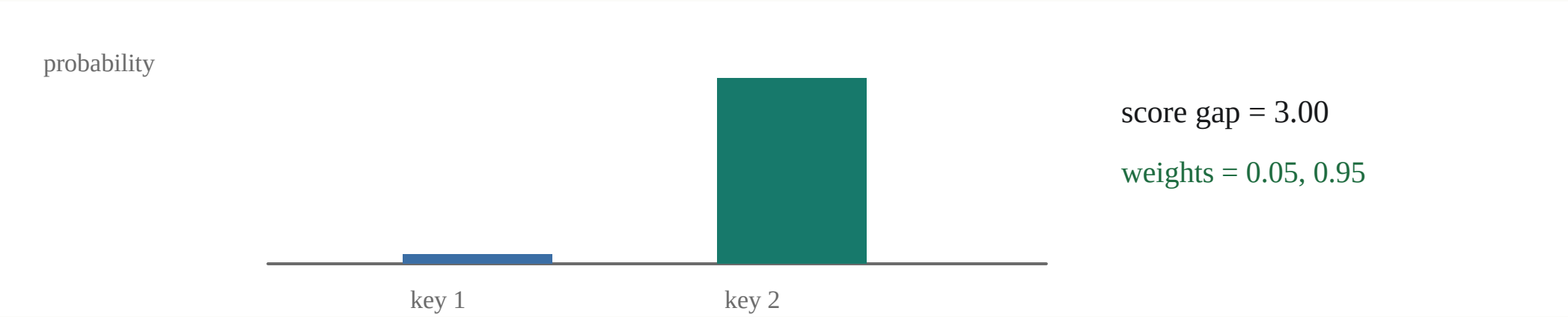
Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

MOTIVATION

Scaled dot-product attention

Why divide by the square root?

Wider query-key vectors can produce more variable dot products. Very unequal scores make softmax nearly one-hot.



Scaling a fixed score gap changes the softmax weights.

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Scaled dot-product attention

Variance of one product

$$\mathbb{E}[q_r k_r] = 0, \quad \mathrm{Var}(q_r k_r) = \mathbb{E}[q_r^2]\mathbb{E}[k_r^2] = 1$$

q_r, k_r independent, mean 0, variance 1

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Scaled dot-product attention

Variance of the sum

Assume all query and key coordinates are mutually independent, with mean zero and variance one.

$$\operatorname{Var}(q^T k) = d_k, \qquad \operatorname{Var}\left(\frac{q^T k}{\sqrt{d_k}}\right) = 1$$

$q, k; d_k$ query/key columns; number of coordinates

This simplifying model motivates the scale.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Scaled dot-product attention

The scale is determined by the assumptions

All coordinates of q, k are mutually independent, with mean 0 and variance 1.

For $u = q^T k$ with $d_k = 16$, find the unscaled standard deviation.

A 16

B 4

C 1

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DERIVATION

Stable attention and masks

The largest logit can be subtracted

$$\frac{e^{s_j - c}}{\sum_\ell e^{s_\ell - c}} = \frac{e^{-c} e^{s_j}}{e^{-c} \sum_\ell e^{s_\ell}} = p_j$$

s_j, p_j finite logit; its softmax weight

c same finite shift for the whole row

Blanchard, Higham & Higham (2021), §4

EXAMPLE

Stable attention and masks

A large-logit example

$$s = (1000, 1000 + \log 3), \quad s - \max s = (-\log 3, 0)$$

s one score row

$e^{s-\max s} = (1/3, 1)$; normalize to $(1/4, 3/4)$.

Blanchard, Higham & Higham (2021), §4 · Course example

EXERCISE

Stable attention and masks

A shared shift or a change of scale?

A score row is $s = (0, \log 3)$. Which change preserves its softmax weights?

- **A** Subtract 5 from both entries
- **B** Multiply both entries by 2
- **C** Add 5 only to the second entry

Blanchard, Higham & Higham (2021), §4 · Course example

DEFINITION

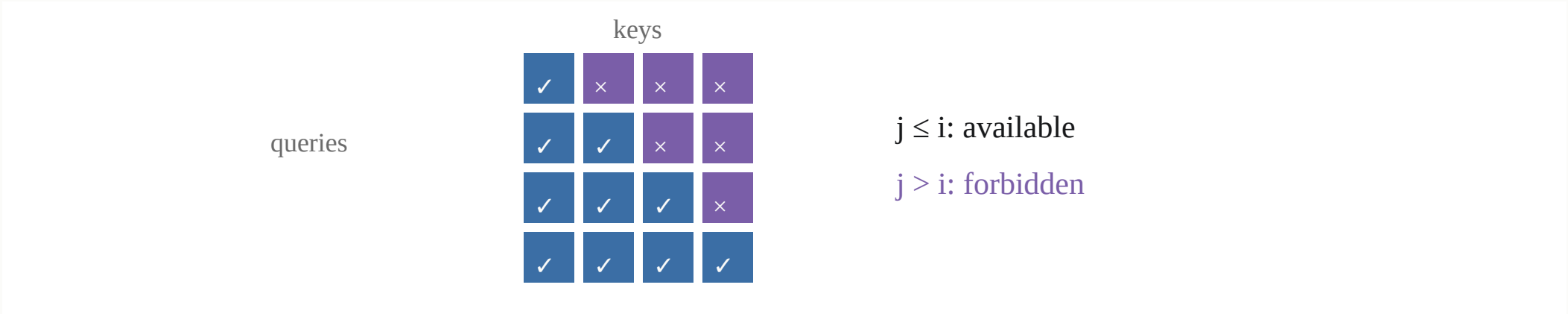
Stable attention and masks

A causal mask forbids future keys

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}, \qquad P = \mathrm{softmax}_{\mathrm{row}}(S + M)$$

i, j query/key positions in ordered self-attention

S, M, P scaled scores; additive mask; row weights



Rows are queries; columns are keys. The upper triangle is forbidden.

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Stable attention and masks

Padding and causality answer different questions

Padding equalizes lengths with placeholders. A key-validity mask excludes them.

$$S : [B, H, T_q, T_k], \quad V_k : [B, 1, 1, T_k], \quad C : [1, 1, T_q, T_k]$$

B, H	batch items; parallel attention calculations (heads)
T_q, T_k	query and key counts
V_k, C	key-valid and causal masks; True allows

Use $V_k \wedge C$. Exclude padded queries from loss.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Stable attention and masks

Mask before normalizing

Scores are $(0, \log 3)$; the second key is forbidden. Which output weights are correct?

A $(1, 0)$

B $(1/4, 0)$

C $(1/4, 3/4)$

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

COUNTEREXAMPLE

Stable attention and masks

Every valid query needs an allowed key

If every masked logit is $-\infty$, the softmax denominator is zero. Subtracting the row maximum also asks for $-\infty - (-\infty)$.

A valid query needs an allowed key. An invalid padded query can use zero output and no loss.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Stable attention and masks

Diagnose the NaN without changing the task

A padded query has no allowed key. Stable softmax returns NaN (Not a Number). Which repair preserves the declared information rule?

A. Use uniform weights over all keys

B. Mark the query invalid and output zero

C. Remove all masks for the batch

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXAMPLE

Stable attention and masks

One attention query, end to end

$$q = (\sqrt{2}, 0)^T, \quad K = \begin{bmatrix} 0 & 1 \\ \log 3 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$$

$q; d_k = 2$	query column; coordinates per query/key
$K, V; T_k = 2$	key/value rows; two pairs, both permitted

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

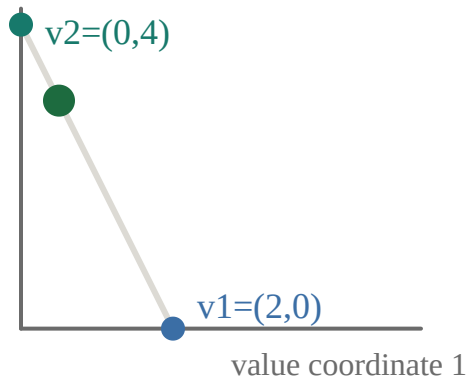
EXAMPLE

Stable attention and masks

The output follows the same data path

$$s = (0, \log 3) \Rightarrow p = (1/4, 3/4) \Rightarrow y = (1/2, 3)^T$$

- s scaled scores
- p row weights
- y mixture of $(2,0)^T$ and $(0,4)^T$



weight of v2 = 0.75

y = (0.50, 3.00)

Weights (1/4, 3/4) place the output at (1/2, 3).

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Stable attention and masks

Transfer: change the permitted information

$d_k = 2$, $q = (\sqrt{2}, 0)^T$, $k_1 = (0, 1)^T$, $k_2 = (\log 3, 0)^T$; $v_1 = (2, 0)^T$, $v_2 = (0, 4)^T$. Only key 1 is allowed. Find the output.

A $(1/2, 3)^T$

B $(2, 0)^T$

C $(0, 0)^T$

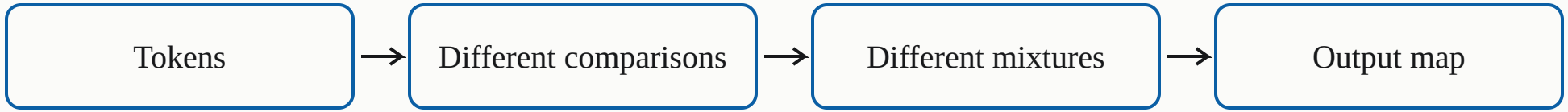
Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

MOTIVATION

Multi-head attention

One mixture can hide useful differences

Each attention head has its own learned query, key and value maps. Multi-head attention (MHA) keeps several mixtures per query, concatenates them, then applies an output map.



Vaswani et al. (2017), §3.2.2

DEFINITION

Multi-head attention

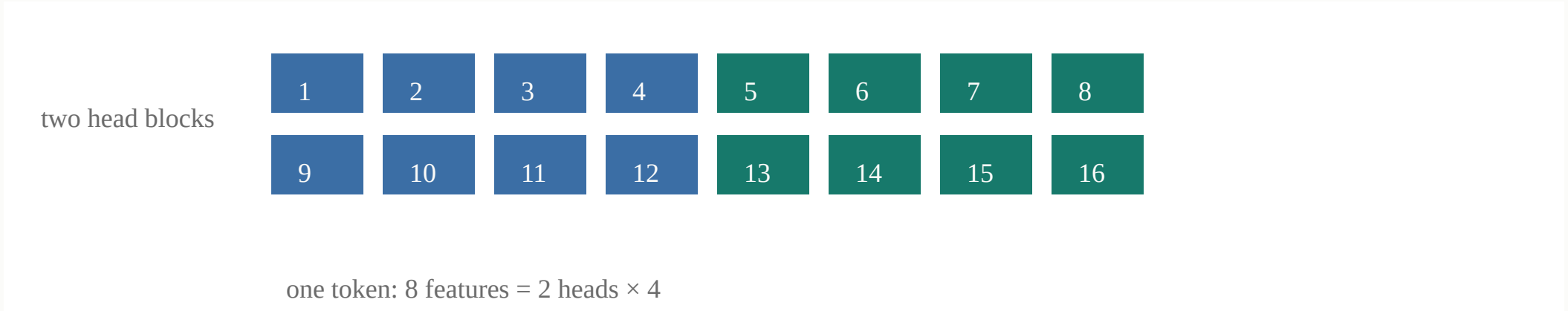
A feature axis can be split into heads

$$Q^{\text{pack}} : [B, T, D] \to Q^{\text{split}} : [B, T, H, d_h], \quad D = H d_h$$

B, T, D batch size; tokens; packed feature width

H, d_h heads; common query/key/value width $d_k = d_v = d_h$

```
Qsplit = Qpack.reshape(B,T,H,dh)
```



A packed feature row splits into two head blocks.

Vaswani et al. (2017), §3.2.2

DERIVATION

Multi-head attention

Transpose puts matrix axes last

$$Q^{\text{split}} : [B, T, H, d_h] \to Q : [B, H, T, d_h], \quad S : [B, H, T, T]$$

B, H	batch items; heads
T, d_h	tokens; coordinates within a head

Apply the same token/head axis swap to K and V.

Vaswani et al. (2017), §3.2.2

EXERCISE

Multi-head attention

Trace a tensor through the heads

For self-attention, $B = 2$, $T = 3$, $D = 8$, $H = 2$. Find the query shape Q after splitting and transposing, and the score shape S.

A Q [2,2,3,4]; S [2,2,3,3]

B Q [2,3,2,4]; S [2,3,2,2]

C Q [2,2,3,8]; S [2,2,8,8]

Vaswani et al. (2017), §3.2.2 · Course example

DEFINITION

Multi-head attention

Concatenate, then mix across heads

For one sequence, use equal key/value width $d_k = d_v = d_h$ in each of H heads.

$$C = \mathrm{Concat}(Y^{(1)}, \ldots, Y^{(H)}), \quad Y = CW_O^T$$

C, W_O, Y T × Hd_h joined heads; D × Hd_h learned map; T × D output

```
heads: [B,H,T,dh]
C = heads.transpose(1,2).reshape(B,T,H*dh)
out = C @ Wo.T
```

Vaswani et al. (2017), §3.2.2

EXAMPLE

Multi-head attention

Two heads, one explicit output map

$$c = (1, 2, 3, -1)^T, \quad W_O = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

c joined head columns: (1,2) and (3,−1)

W_O 2×4 output map

$W_O c = (1 + 3, 2 - 1)^T = (4, 1)^T.$

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Multi-head attention

Transfer: a different output map

The joined head vector is $c = (1, 2, 3, -1)^T$. The output map is $W_O = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & -1 \end{bmatrix}$. Find $W_O c$.

A $(3, 2)^T$

B $(4, 1)^T$

C $(1, 2, 3, -1)^T$

Vaswani et al. (2017), §3.2.2 · Course example

Multi-head attention

The attention core follows the equations

Pseudocode for valid queries:

`Q,K,V: [B,H,T,dh]`; equal key/value width d_h.

```
S = Q @ K.transpose(-2,-1) / sqrt(dh)
S = masked_fill(S, forbidden, -inf)
P = softmax(S, dim=-1)
heads = P @ V
```

True in `forbidden` blocks a key. Every valid query needs a key. Handle padded query rows separately: zero their outputs and omit their loss.

Vaswani et al. (2017), §3.2.2

EXERCISE

Multi-head attention

Correct shapes can hide a wrong normalization

Scores have shape [B,H,Tq,Tk]: batch, heads, queries, keys. A programmer uses `softmax(S,dim=-2)`. What is normalized?

A Keys for each query

B Queries for each key

C Features of each token

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Multi-head attention

Transfer: broadcast a key-validity mask

Scores have shape [2,3,4,5]: batch, heads, queries, keys. Key validity has shape [2,5], with True for allowed keys. Which reshape shares this key rule across every head and query?

A [2,1,1,5]

B [2,5,1,1]

C [1,1,2,5]

Vaswani et al. (2017), §3.2.2 · Course example

DERIVATION

Position and order

Content-only attention follows a permutation

Permutation equivariance: reordering input tokens reorders the output vectors the same way.

$$\mathrm{Attn}(\Pi X) = \Pi\,\mathrm{Attn}(X)$$

X, Π T × D token rows; T × T row-permutation matrix

Shared projections, no position features, no fixed mask, and dropout disabled.

Vaswani et al. (2017), §3.5

EXERCISE

Position and order

What does content alone fail to identify?

The same two feature vectors are presented in opposite orders to unmasked attention with no position features.

A. Dot products recover their original order

B. Outputs move with the input permutation

C. Every output becomes zero

Vaswani et al. (2017), §3.5 · Course example

DEFINITION

Position and order

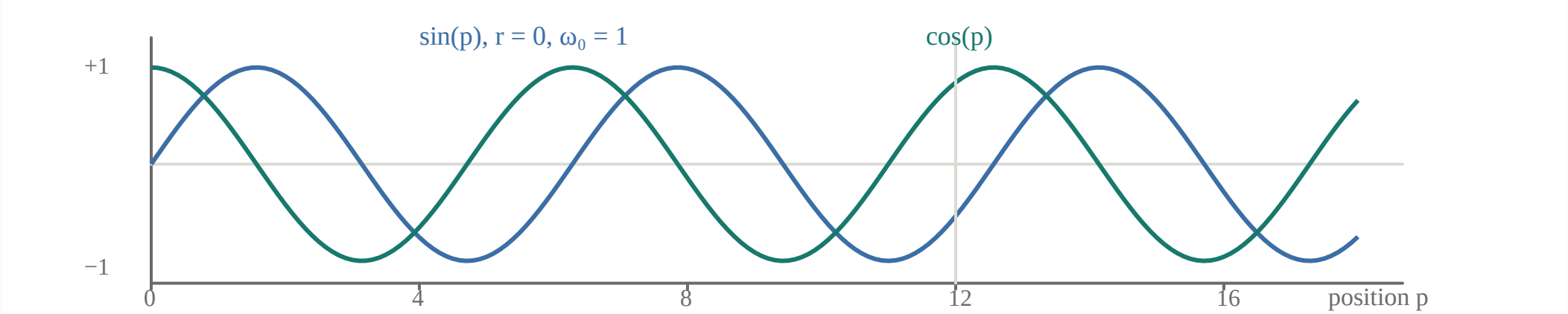
Sinusoids give each position a pattern

For token features x_p, add the position vector: $\tilde{x}_p = x_p + \mathrm{PE}_p$.

$$\mathrm{PE}_{p,2r} = \sin(p\omega_r), \quad \mathrm{PE}_{p,2r+1} = \cos(p\omega_r)$$

p, r, D position; coordinate-pair index 0,…,D/2−1; even width

$\omega_r = 10000^{-2r/D}$ frequency of pair r



One sine/cosine pair across token positions.

Vaswani et al. (2017), §3.5

DERIVATION

Position and order

A shift follows angle-addition identities

$$\begin{bmatrix}\sin((p+\Delta)\omega)\\ \cos((p+\Delta)\omega)\end{bmatrix} = \begin{bmatrix}\cos(\Delta\omega) & \sin(\Delta\omega)\\ -\sin(\Delta\omega) & \cos(\Delta\omega)\end{bmatrix}\begin{bmatrix}\sin(p\omega)\\ \cos(p\omega)\end{bmatrix}$$

p, Δ, ω starting position, position shift, one fixed pair frequency

Vaswani et al. (2017), §3.5

EXERCISE

Position and order

Transfer: one sinusoidal pair at a new position

One position pair is $(\sin(p\omega), \cos(p\omega))$. For this illustrative frequency $\omega = \pi/2$, move from position p=0 to p=1.

A (1,0)

B (0,1)

C (−1,0)

Vaswani et al. (2017), §3.5 · Course example

DEFINITION

Position and order

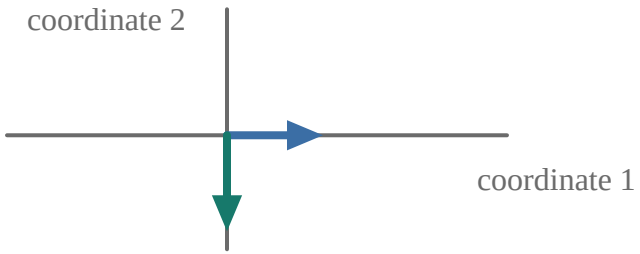
Rotary embeddings act inside the comparison

Rotary position embedding (RoPE)

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$R(\theta)$ 2×2 counterclockwise rotation

$\tilde{q}_p = R(p\omega)q_p, \widetilde{k}_s = R(s\omega)k_s$. q,k: query/key pairs; p,s: positions; ω: frequency.



unit length after rotation

angle = 4.71 rad

A query pair rotates with position while keeping its length.

Su et al., RoFormer (2021), §3.2

DERIVATION

Position and order

Relative positions appear in the dot product

$$\tilde{q}_p^T \widetilde{k}_s = q_p^T R(p\omega)^T R(s\omega) k_s = q_p^T R((s-p)\omega) k_s$$

q_p, k_s	unrotated two-coordinate query/key columns at positions p,s
$R, \omega, \tilde{q}_p, \widetilde{k}_s$	rotation; fixed frequency; position-rotated pairs
$s - p$	key position relative to the query

Su et al., RoFormer (2021), §3.2

EXAMPLE

Position and order

A quarter-turn example

$$q = k = (1, 0)^T, \quad p = 0, \quad s = 1, \quad \omega = \pi/2$$

q, k unrotated query/key coordinate pairs

p, s, ω query/key positions; illustrative frequency

After rotation: $\tilde{q} = (1, 0)^T$, $\tilde{k} = (0, 1)^T$. Their dot product is 0.

Su et al., RoFormer (2021), §3.2 · Course example

EXERCISE

Position and order

Transfer: shift both positions

Unrotated pairs: $q = k = (1, 0)^T$; frequency $\omega = \pi/2$. Change query/key positions (p, s) from $(0, 1)$ to $(2, 3)$. Find the rotated query–key dot product before attention scaling.

A 0

B 1

C -1

Su et al., RoFormer (2021), §3.2 · Course example

DEFINITION

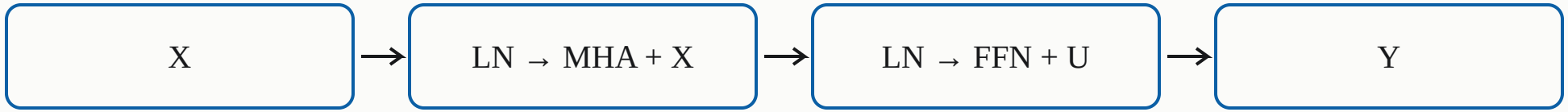
Transformer blocks

The pre-LN block has two residual paths

Residual connections add branch outputs to inputs. Pre-LN normalizes before each branch.

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \quad Y = U + \mathrm{FFN}(\mathrm{LN}(U))$$

X, U, Y	T × D representations
MHA, FFN	multi-head attention; feed-forward network
LN	layer normalization within each token



Xiong et al. (2020), §3.1, Table 1

RECALL

Transformer blocks

LayerNorm uses the token feature axis

$$\widehat{x}_r = \frac{x_r - \mu}{\sqrt{v + \epsilon}}, \quad \mu = \frac{1}{D}\sum_{r=1}^{D} x_r, \quad v = \frac{1}{D}\sum_{r=1}^{D}(x_r - \mu)^2$$

x, r, D	one token; feature index; feature count
μ, v	feature mean; mean squared deviation
$\epsilon > 0$	stability constant

Output: $\gamma \odot \widehat{x} + \beta$. Learned scale γ and shift β each have D coordinates; $\odot$ acts coordinatewise.

Xiong et al. (2020), §3.1, Table 1

EXERCISE

Transformer blocks

Transfer: normalize one token

One token is $x = (1, 3)$. Use $\gamma = (1, 1)$, $\beta = (0, 0)$ and $\epsilon = 0$ for this nonconstant numerical example.

A (−1,1)

B (−1/√2, 1/√2)

C (1,3)

Xiong et al. (2020), §3.1, Table 1 · Course example

DEFINITION

Transformer blocks

The feed-forward branch transforms each token

A positionwise feed-forward network uses the same nonlinear feature map separately on each token.

$$f(x) = W_2\phi(W_1x + b_1) + b_2$$

x, W_1, W_2	D-feature token; F × D and D × F maps
ϕ, b_1, b_2	coordinatewise nonlinearity; F- and D-coordinate biases

F is the hidden width, usually larger than D.

Vaswani et al. (2017), §3.3

EXERCISE

Transformer blocks

A residual connection constrains the shape

The residual input X has shape [2,5,12]: batch, tokens, features. Its attention branch returns [2,5,8]. Can they be directly added?

A. Yes, ordinary broadcasting repairs it

B. No, the branch must return width 12

C. Yes, because both have five tokens

Xiong et al. (2020), §3.1, Table 1 · Course example

EXERCISE

Transformer blocks

Transfer: a missing path changes the architecture

$Y = X + FFN(MHA(X))$ is described as the pre-LN block. Here MHA is multi-head attention and FFN is the tokenwise feed-forward network. What is missing?

A. Only position information

B. Both normalizations and the intermediate residual

C. A final softmax only

Xiong et al. (2020), §3.1, Table 1 · Course example

DERIVATION

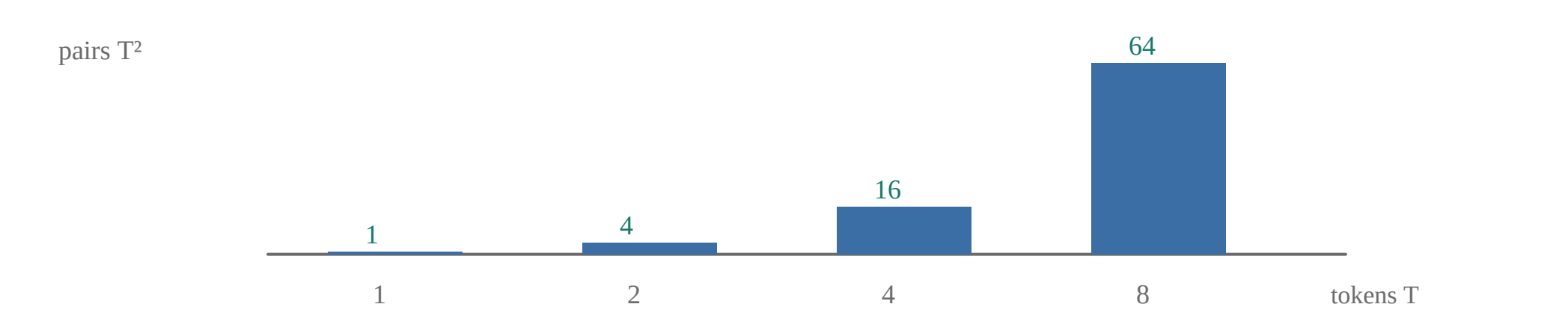
Cost and the KV cache

Pairwise comparison creates quadratic work

$$QK^T : O(T^2 d_h), \quad PV : O(T^2 d_h), \quad Hd_h = D$$

Q, K, V, P	queries, keys, values, row-softmax weights
T, H, d_h, D	tokens, heads, equal key/value head width, total width

Per sequence/layer: mixing $O(T^2 D)$; projections $O(TD^2)$.



Query-key pair counts grow quadratically with token length.

Vaswani et al. (2017), §3.2.2

EXAMPLE

Cost and the KV cache

Memory follows the score shape

$$B = 1,\ H = 8,\ T = 1024: \quad BHT^2 = 8{,}388{,}608$$

B, H, T batch items, heads, query/key tokens

At 2 bytes per entry: 16 MiB for one materialized score tensor. 1 MiB = 2^{20} bytes. Other tensors are excluded.

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Cost and the KV cache

Transfer: a longer sequence

The materialized score tensor has shape [B,H,T,T]: batch, heads, queries, keys.

Keep B, H and bytes per entry fixed. Double T. How does its memory change?

A ×2

B ×4

C Unchanged

Vaswani et al. (2017), §3.2.2 · Course example

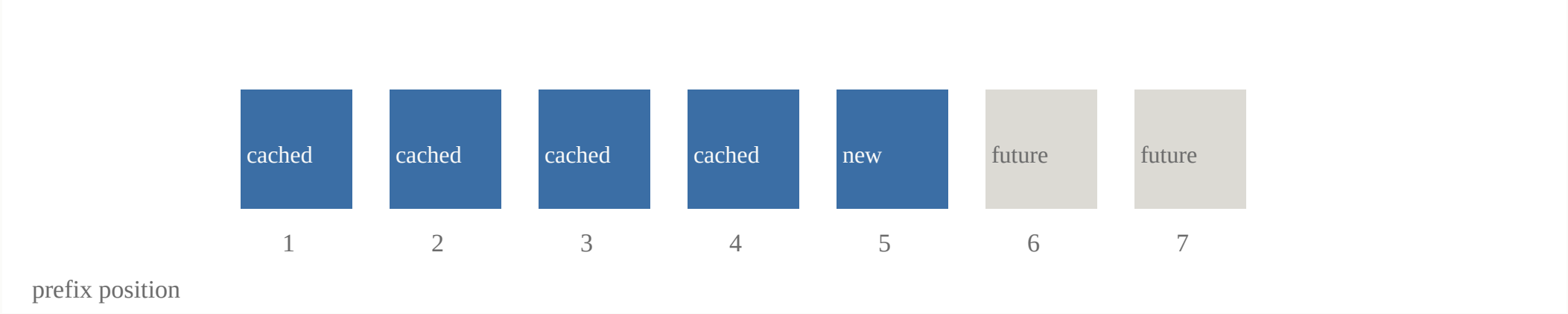
DEFINITION

Cost and the KV cache

Unchanged causal prefixes permit reuse

Autoregressive decoding appends one token at a time. A key-value (KV) cache stores earlier key/value projections.

Exact reuse needs causal dependencies, fixed earlier inputs/positions/parameters, and deterministic inference with dropout disabled.



An unchanged prefix stays cached while one new row is appended.

Shazeer (2019), §2.4–2.4.1

DERIVATION

Cost and the KV cache

One new query reads all cached keys

$$q_t^T K_{1:t}^T : \ [1, d_h][d_h, t] = [1, t]$$

$q_t, K_{1:t}$	new query column; t key rows including the new key
d_h, D	equal key/value head width; total width

Mixing $O(tD)$ per step; KV storage $O(tD)$ per layer and batch item.

```
K = concat([Kold, knew], axis=-2)
V = concat([Vold, vnew], axis=-2)
```

Shazeer (2019), §2.4–2.4.1

EXERCISE

Cost and the KV cache

Transfer: can this memory be reused?

Camera tokens are replaced by a new frame. Can their old cross-attention keys and values serve as exact representations of the new frame?

A Yes, when shapes match

B No, recompute the changed memory

C Yes, if action horizon is fixed

Shazeer (2019), §2.4–2.4.1 · Course example

EXAMPLE

Cost and the KV cache

Cache validity and cumulative decoding cost

$$\sum_{t=1}^{T} t = \frac{T(T+1)}{2}, \quad \sum_{t=1}^{T} t^2 = \frac{T(T+1)(2T+1)}{6}$$

t, T current/final prefix length

Dense score dot products before masking: cached decoding sums t; full-prefix reruns sum t². Mixing costs: $O(T^2 D)$ versus $O(T^3 D)$.

Shazeer (2019), §2.4–2.4.1 · Course example

EXERCISE

Cost and the KV cache

Transfer: count incremental comparisons

Generate four tokens from an empty prefix, one head. Count dense query–key dot products before masking, including each current token. Compare cached decoding with a full t × t score matrix recomputed at every step.

A Cached 10; recomputed 30

B Cached 4; recomputed 16

C Cached 16; recomputed 16

Shazeer (2019), §2.4–2.4.1 · Course example

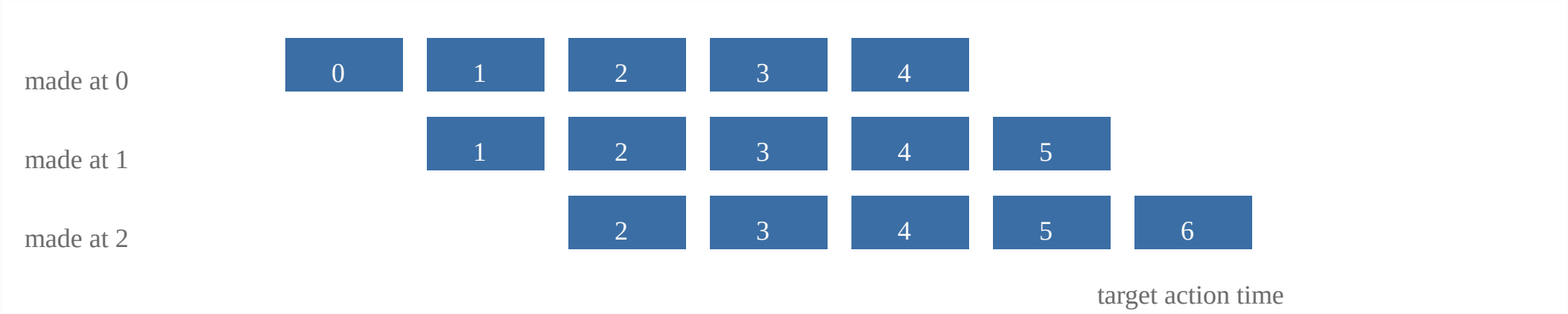
MOTIVATION

Action chunking

A coordinated movement spans several decisions

Action chunking predicts several consecutive future motor targets together from the current observation.

The robot may execute a prefix, observe again, and replan. o_t: observation now; K: predicted chunk length.



Each row is a chunk prediction; columns are target action times.

Zhao et al. (2023), §IV-A

DEFINITION

Action chunking

Prediction time and target time are different indices

$$\widehat{A}_t \in \mathbb{R}^{K \times d_a}, \quad \widehat{A}_t[j, :] = \hat{a}_{t+j|t}^T, \quad 0 \le j < K$$

t, j, K, d_a prediction time; row offset; chunk length; action dimension

$\hat{a}_{t+j|t}, \widehat{A}_t$ action column for target t+j predicted at t; row-stored chunk

Zhao et al. (2023), §IV-A

EXERCISE

Action chunking

Chunk length is not execution length

At prediction time t, a policy predicts K=6 actions and executes the first E=2 before observing again. K is chunk length; E is execution-prefix length. Which target times from this chunk are executed?

A. Times t through t+5

B. Times t and t+1

C. Only time t+5

Zhao et al. (2023), §IV-A · Course example

DEFINITION

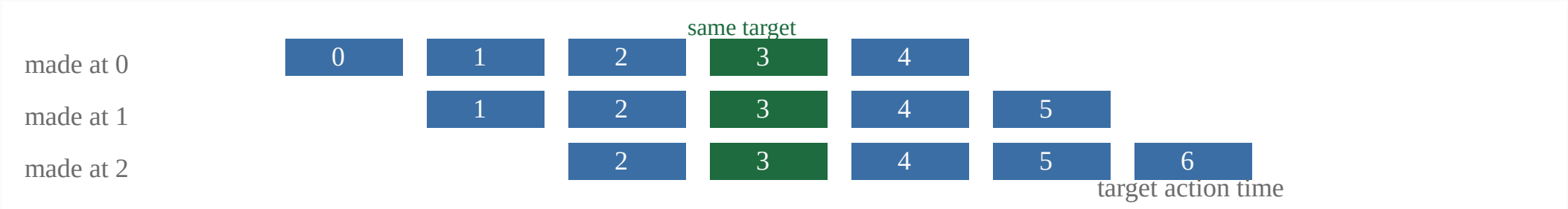
Temporal ensembling

Overlapping chunks predict the same target

Chunks cover target s: $r_j \le s < r_j + K$.

$$\bar{a}_s = \frac{\sum_{j=0}^{J-1} w_j \hat{a}_{s|r_j}}{\sum_{j=0}^{J-1} w_j}, \qquad w_j = e^{-mj}$$

s, r_j	target, prediction times
J, K, w_j, m	chunks, oldest first; actions/chunk; weight; decay $m \ge 0$
$\hat{a}_{s\|r_j}, \bar{a}_s$	predicted/combined action



One target column across overlapping chunks.

Zhao et al. (2023), §IV-A

EXAMPLE

Temporal ensembling

One target, two predictions

$$\hat{a}_{s|r_0} = 2, \quad \hat{a}_{s|r_1} = 4, \quad m = \log 2$$

$r_0 < r_1, s$ oldest/newest prediction times; common target time

$\hat{a}_{s|r_j}, w_j$ scalar action prediction; weight e^{-mj}

$m, \bar{a}_s$ decay parameter; combined action for target s

$w = (1, 1/2)$, so $\bar{a}_s = (2 + 2)/(3/2) = 8/3$.

Zhao et al. (2023), §IV-A · Course example

EXERCISE

Temporal ensembling

Transfer: three estimates for one target

Oldest-to-newest scalar predictions for the same target s are $1, 3, 5$. Use $w_j = e^{-mj}$ with $j = 0, 1, 2$ and $m = \log 2$. Find the normalized ensemble.

A 3

B 15/7

C 27/7

Zhao et al. (2023), §IV-A · Course example

INTERPRETATION

Temporal ensembling

Averages need compatible action representations

Average predictions for the same target time in the same coordinate system.

Smoothing alone does not enforce collision, dynamics or joint-limit constraints.

Zhao et al. (2023), §IV-A

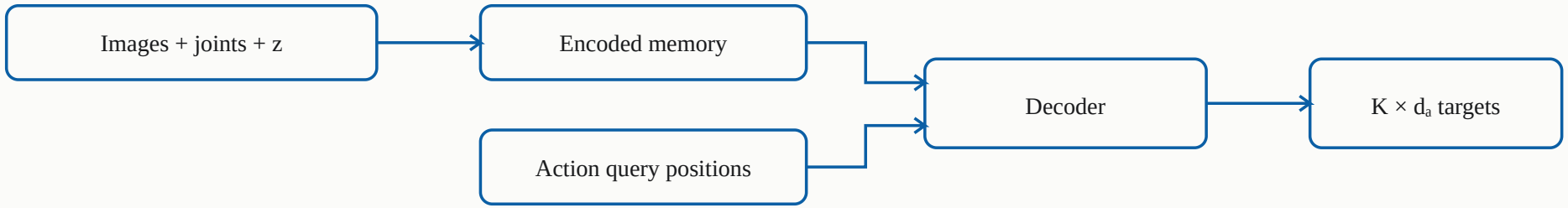
DEFINITION

ACT: observations to actions

ACT combines observation tokens with action queries

Action Chunking with Transformers (ACT) predicts K motor targets.

o_t, z	current images/joints; latent input for demonstration variation
E_t	encoded observation memory
$K, d_a, \hat{A}_t$	chunk length; action width; predicted $K \times d_a$ chunk



Zhao et al. (2023), §IV-B–C, Appendix C

EXAMPLE

ACT: observations to actions

A concrete observation-token count

$$4(15 \cdot 20) + 2 = 1202$$

4	camera views in the ACT paper
15×20	feature grid per view
$+2$	joint-state and latent tokens

Each token has 512 features in the reported model. Spatial positions accompany image features.

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

DEFINITION

ACT: observations to actions

Action queries use cross-attention

$$S_a = \frac{Q_a K_o^T}{\sqrt{d_k}} \in \mathbb{R}^{K \times T_o}$$

$Q_a \in \mathbb{R}^{K \times d_k}$	one head's projected action queries
$K_o \in \mathbb{R}^{T_o \times d_k}$	projected observation keys
K, T_o, d_k	action slots; observation tokens; query/key width
S_a	action-to-observation scores before softmax

Zhao et al. (2023), §IV-B–C, Appendix C

EXERCISE

ACT: observations to actions

Transfer: encoder tokens and action slots

A model has 20 observation tokens and predicts a chunk of 6 actions. Each action query reads every observation token. What is one head’s cross-attention score shape?

A. 20×20

B. 6×20

C. 6×6

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

MOTIVATION

ACT: latent-variable training

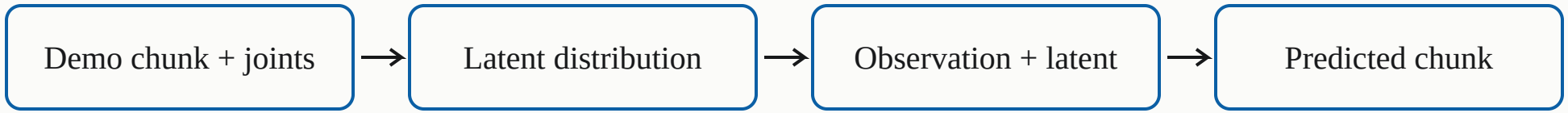
Demonstrations can differ at the same observation

Conditional variational autoencoder (CVAE)

$$q_\phi(z \mid A_t, r_t), \qquad p_\theta(A_t \mid o_t, z)$$

A_t, r_t, o_t demonstrated chunk; observed joints; images and joints

z, ϕ, θ latent vector; encoder and decoder parameters



Zhao et al. (2023), §IV-B–C, Appendix C

DEFINITION

ACT: latent-variable training

The encoder predicts a diagonal Gaussian

$$q_\phi(z \mid A_t, r_t) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2)), \quad p(z) = \mathcal{N}(0, I)$$

`z, μ, σ`	sample, mean, positive standard deviations
`L, j`	latent dimension; coordinate index
`qφ, p`	encoder density; standard-normal prior
`Aₜ, rₜ, φ`	target chunk, joints; encoder parameters

diag(σ²): L×L covariance. Output $\ell_j = \log \sigma_j^2$; $\sigma_j = e^{\ell_j/2}$.

Kingma & Welling (2014), §2, Appendix B

DERIVATION

ACT: latent-variable training

Reparameterization exposes a differentiable path

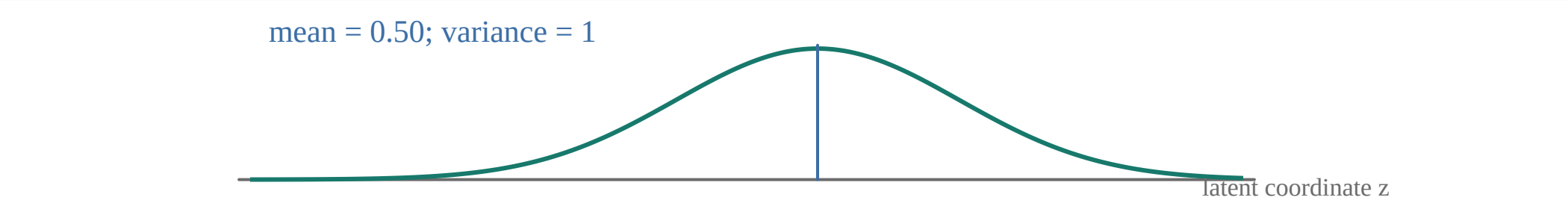
Draw ε independently of encoder parameters.

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu + \sigma \odot \epsilon$$

$z, \epsilon \in \mathbb{R}^L$ sample; standard-normal noise

$\mu, \sigma \in \mathbb{R}^L$ mean; positive standard deviations

$\odot$ coordinatewise multiplication



Changing the mean shifts a Gaussian without changing its width.

Kingma & Welling (2014), §2, Appendix B

EXERCISE

ACT: latent-variable training

Log-variance is not the standard deviation

One latent coordinate has $\mu = 1$, $\ell = \log \sigma^2 = \log 4$, and $\epsilon = -1$. Use $z = \mu + \sigma\epsilon$. Find z.

A -3

B -1

C 1

Kingma & Welling (2014), §2, Appendix B · Course example

DERIVATION

ACT: latent-variable training

A bound makes the latent objective tractable

$$p(A|o) = \int p(A, z|o) dz$$

$$\mathbb{E}_q \log \frac{p(A, z|o)}{q(z)} = \log p(A|o) - D_{KL}(q \| p(z|A, o))$$

`A, o, z`	chunk, observation, latent
`p≡pθ; q`	fixed trained model; encoder qφ(z|A,r), φ weights
`r, E_q`	joints; average over z∼q

$D_{KL}(q\|p_*) = \mathbb{E}_q \log(q/p_*)$; p_*: density. Positive densities; finite expectations.

Kingma & Welling (2014), §2, Appendix B

DERIVATION

ACT: latent-variable training

Nonnegative KL gives the evidence lower bound

$$\begin{aligned}\mathcal{L}_{ELBO} &= \mathbb{E}_q \log \frac{p(A|o,z)p(z)}{q(z)} \\ &= \mathbb{E}_q \log p(A|o,z) - D_{KL}(q\|p) \\ &\leq \log p(A|o)\end{aligned}$$

`L_ELBO`	evidence lower bound; maximize
`A, o, z`	chunk, observation, latent
`q; p(z)`	encoder qφ(z|A,r); independent prior
`p≡pθ; p(A|o,z)`	fixed trained model; decoder density; φ weights, r joints

Kingma & Welling (2014), §2, Appendix B

EXERCISE

ACT: latent-variable training

Transfer: which divergence creates the gap?

For a fixed chunk A and observation o, let $q(z)$ be the encoder density. The exact identity is $\log p_\theta(A \mid o) = \mathcal{L}_{ELBO} + G$. Which G is the gap?

A $D_{KL}(q \| p_\theta(z \mid A, o))$

B $-D_{KL}(q \| p(z))$

C The action prediction error alone

Kingma & Welling (2014), §2, Appendix B · Course example

DEFINITION

ACT: latent-variable training

ACT balances reconstruction and latent regularization

$$\mathcal{L}_{train} = \frac{1}{K d_a} \sum_{j=0}^{K-1} \sum_{c=1}^{d_a} |\hat{A}_{jc} - A_{jc}| + \beta D_{KL}(q_\phi \| \mathcal{N}(0, I))$$

$A, \hat{A} \in \mathbb{R}^{K \times d_a}$	target/predicted chunks; all valid
$K, d_a;\ j, c$	chunk length, action width; row/coordinate indices
q_ϕ, β	encoder latent density; KL weight $\beta \geq 0$

Zhao et al. (2023), §IV-B–C, Appendix C

DERIVATION

ACT: latent-variable training

Gaussian KL follows from a log-density ratio

$$\log \frac{q_j(z_j)}{p_j(z_j)} = -\log \sigma_j - \frac{(z_j - \mu_j)^2}{2\sigma_j^2} + \frac{z_j^2}{2}$$

q_j, p_j	$\mathcal{N}(\mu_j, \sigma_j^2)$ and $\mathcal{N}(0, 1)$ densities
z_j, μ_j	latent coordinate and its mean
$\sigma_j > 0$	standard deviation

Use $\mathbb{E}_q[(z_j - \mu_j)^2] = \sigma_j^2$ and $\mathbb{E}_q[z_j^2] = \mu_j^2 + \sigma_j^2$.

Kingma & Welling (2014), §2, Appendix B

DERIVATION

ACT: latent-variable training

The diagonal-Gaussian KL is explicit

$$\mathbb{E}_q \log(q_j/p_j) = -\log\sigma_j - \frac{1}{2} + \frac{1}{2}(\mu_j^2 + \sigma_j^2)$$

$$D_{KL}(q\|p) = \frac{1}{2}\sum_{j=1}^{L}(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$$

q, p	diagonal-Gaussian encoder; $\mathcal{N}(0, I)$ prior
L, j	latent dimension; coordinate index
μ_j, σ_j^2	coordinate mean and positive variance

Kingma & Welling (2014), §2, Appendix B

EXAMPLE

ACT: latent-variable training

A latent penalty calculated by hand

Compare a two-coordinate Gaussian encoder with the prior $p = \mathcal{N}(0, I)$.

$$\mu = (1, 0)^T, \quad \sigma^2 = (1, 4)^T$$

μ, σ^2 encoder means and coordinate variances

$D_{KL}(q\|p) = \frac{1}{2}[1 + (4 - 1 - \log 4)] = 2 - \log 2 \approx 1.3069.$

Kingma & Welling (2014), §2, Appendix B · Course example

EXERCISE

ACT: latent-variable training

Transfer: combine the two loss terms

Four valid action-coordinate errors are $1, -1, 0, 2$. A scalar Gaussian encoder has $\mu = 1$, $\sigma^2 = 1$; its prior is $\mathcal{N}(0, 1)$. Use mean L1 and $\beta = 2$. Find $\mathcal{L}_{train}$.

A 1

B 2

C 5

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

DEFINITION

ACT: deployment

The deployed ACT policy fixes the latent to zero

$$\hat{A}_t = f_\theta(o_t, z = 0)$$

o_t, θ	current images/joints; learned policy parameters
$z = 0$	zero vector, the standard-Gaussian prior mean
f_θ	learned chunk-prediction policy
$\hat{A}_t$	predicted action chunk

The demonstration-conditioned latent encoder is discarded at inference.

Zhao et al. (2023), §IV-B–C, Appendix C

EXERCISE

ACT: deployment

Training and inference use different information

The robot receives current images and joint measurements. Which input route matches the reported ACT inference procedure?

A. Encode the unknown future target chunk

B. Current observations plus z=0

C. Draw z from the demonstration posterior

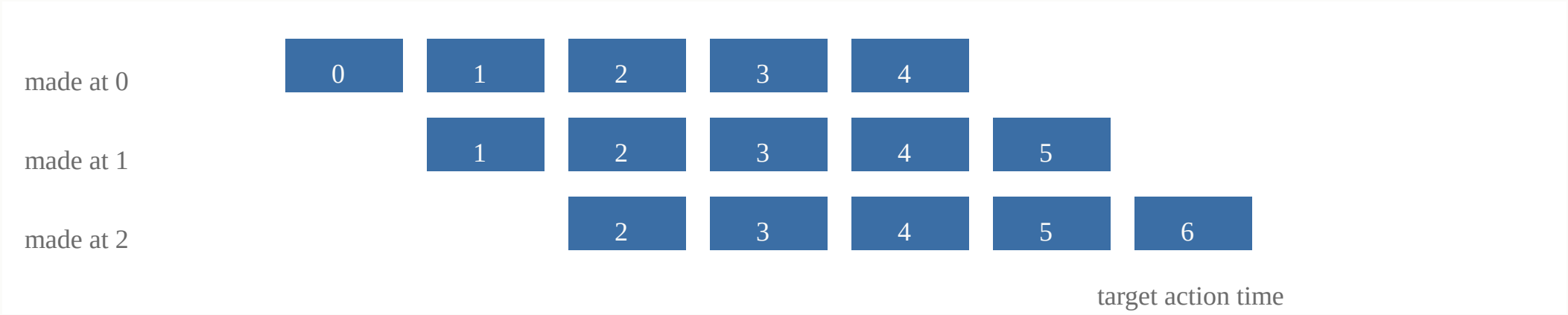
Zhao et al. (2023), §IV-B–C, Appendix C · Course example

SYNTHESIS

ACT: deployment

The complete robot-policy loop

Observe, encode, predict a chunk, combine predictions for the current target, execute, then observe again.



Replanning updates future chunks as fresh observations arrive.

Zhao et al. (2023), §IV-B–C, Appendix C

EXERCISE

ACT: deployment

Transfer: trace the live policy loop

At target time s, two chunks predict scalar actions 2 and 4, oldest first. Use $w_j = e^{-mj}$ with $m = \log 2$. A fresh observation arrives after execution. What target is executed, and what computation follows?

A. Execute 8/3; encode the fresh observation

B. Execute 3; change β

C. Execute 4; reuse all old image features

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

EXERCISE

ACT: deployment

Transfer: a changed scene after prediction

A chunk predicts six actions. After one executed action, the object moves.

Which response uses the new observation?

A. Keep executing all old targets without recomputing

B. Encode the new observation and predict a new chunk

C. Change the KL coefficient at inference

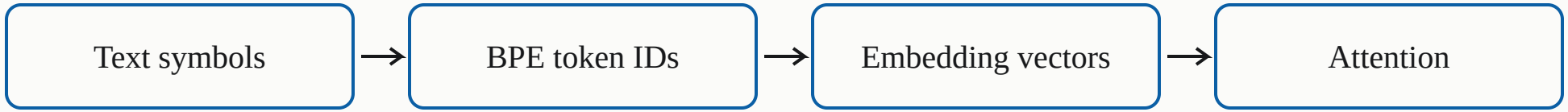
Zhao et al. (2023), §IV-B–C, Appendix C · Course example

BRIDGE

Discrete tokens and continuous actions

Byte-pair encoding is a text representation choice

Byte-pair encoding (BPE) builds a subword vocabulary by repeatedly merging frequent adjacent symbol pairs. A tokenizer maps text to discrete token IDs.



Sennrich, Haddow & Birch (2016), §3.2

DEFINITION

Discrete tokens and continuous actions

Top-p restricts a categorical sampling set

Top-p keeps the smallest sorted prefix reaching mass ρ.

$$\sum_{j=1}^{k-1} p_{(j)} < \rho \leq \sum_{j=1}^{k} p_{(j)}$$

$p_{(1)} \geq p_{(2)} \geq \cdots$	category probabilities sorted largest first
$\rho \in (0, 1], k$	mass threshold; smallest sufficient prefix length; j is sorted rank

Zero excluded mass, renormalize, then sample.

Holtzman et al. (2020), §3.1

EXERCISE

Discrete tokens and continuous actions

Transfer: the retained set depends on mass

Sorted token probabilities are $(0.5, 0.3, 0.2)$ and the top-p threshold is $\rho = 0.7$. Retain the smallest prefix reaching ρ, then renormalize. Which distribution is sampled?

A $(0.5, 0.3, 0.2)$

B $(0.625, 0.375, 0)$

C $(1, 0, 0)$

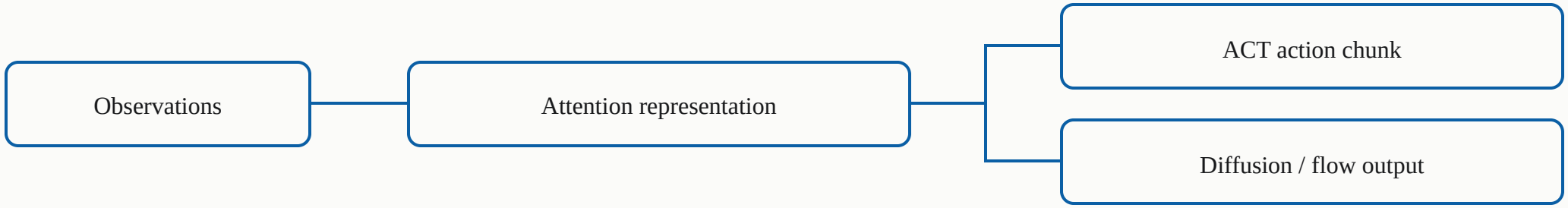
Holtzman et al. (2020), §3.1 · Course example

SYNTHESIS

Transformers and ACT

One representation, several output models

Attention builds a conditional representation. ACT maps it to an action chunk. Diffusion and flow matching will use conditioning to guide iterative generation.



Zhao et al. (2023), §IV-B–C, Appendix C

References

Primary sources and reading 1

1. Vaswani et al. (2017). Attention Is All You Need. §3.2–3.5, Eq. 1; NeurIPS.
2. Blanchard, Higham & Higham (2021). Accurately Computing the Log-Sum-Exp and Softmax Functions. §4.

References

Primary sources and reading 2

1. Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. §3.2.
2. Xiong et al. (2020). On Layer Normalization in the Transformer Architecture. §3.1, Table 1; ICML.
3. Shazeer (2019). Fast Transformer Decoding: One Write-Head is All You Need. §2.4–2.4.1.

References

Primary sources and reading 3

1. Zhao et al. (2023). Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. §IV and Appendix C; RSS.
2. Kingma & Welling (2014). Auto-Encoding Variational Bayes. §2, Appendix B; ICLR.

References

Primary sources and reading 4

1. Sennrich, Haddow & Birch (2016). Neural Machine Translation of Rare Words with Subword Units. §3.2; ACL.
2. Holtzman et al. (2020). The Curious Case of Neural Text Degeneration. §3.1; ICLR.